future internet

IMPACT FACTOR 4.6

CITESCORE 10.0

Article

Deep Reinforcement Learning for Adaptive Robotic Grasping and Post-Grasp Manipulation in Simulated Dynamic Environments

Henrique C. Ferreira and Ramiro S. Barbosa

Special Issue

Artificial Intelligence and Control Systems for Industry 4.0 and 5.0

Edited by

Dr. Filipe Pereira and Prof. Dr. Paulo Leitao

MDPI

https://doi.org/10.3390/fi17100437

future internet

MDPI

Article

Deep Reinforcement Learning for Adaptive Robotic Grasping and Post-Grasp Manipulation in Simulated Dynamic Environments

Henrique C. Ferreira [1] and Ramiro S. Barbosa [1,2,*]

[1] Department of Electrical Engineering, Institute of Engineering—Polytechnic of Porto (ISEP/IPP), 4249-015 Porto, Portugal; 1180528@isep.ipp.pt
[2] GECAD—Research Group on Intelligent Engineering and Computing for Advanced Innovation and Development, ISEP/IPP, 4249-015 Porto, Portugal
* Correspondence: rsb@isep.ipp.pt

Abstract

This article presents a deep reinforcement learning (DRL) approach for adaptive robotic grasping in dynamic environments. We developed UR5GraspingEnv, a PyBullet-based simulation environment integrated with OpenAI Gym, to train a UR5 robotic arm with a Robotiq 2F-85 gripper. Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO) were implemented to learn robust grasping policies for randomly positioned objects. A tailored reward function, combining distance penalties, grasp, and pose rewards, optimizes grasping and post-grasping tasks, enhanced by domain randomization. SAC achieves an 87% grasp success rate and 75% post-grasp success, outperforming PPO 82% and 68%, with stable convergence over 100,000 timesteps. The system addresses post-grasping manipulation and sim-to-real transfer challenges, advancing industrial and assistive applications. Results demonstrate the feasibility of learning stable and goal-driven policies for single-arm robotic manipulation using minimal supervision. Both PPO and SAC yield competitive performance, with SAC exhibiting superior adaptability in cluttered or edge cases. These findings suggest that DRL, when carefully designed and monitored, can support scalable learning in manipulation tasks.

Keywords: robotic grasping; reinforcement learning; deep reinforcement learning (DRL); adaptive manipulation; robotic simulation; proximal policy optimization (PPO); soft actor-critic (SAC); deep deterministic policy gradient (DDPG); autonomous robotics; physics-based simulation

Academic Editor: Paolo Bellavista

Received: 11 August 2025
Revised: 23 September 2025
Accepted: 24 September 2025
Published: 26 September 2025

Citation: Ferreira, H.C.; Barbosa, R.S. Deep Reinforcement Learning for Adaptive Robotic Grasping and Post-Grasp Manipulation in Simulated Dynamic Environments. *Future Internet* **2025**, *17*, 437. https://doi.org/10.3390/fi17100437

1. Introduction

Robotic grasping refers to the autonomous ability of a robotic manipulator to detect, plan, and execute actions that enable secure physical interaction with objects of various geometries and materials. In unstructured environments, this task becomes significantly more challenging due to the unpredictability of object placement, occlusions, dynamic disturbances, and variability in object properties. A successful grasp must also consider stability over time, task relevance (e.g. correct pose for assembly), and minimal risk of collision or slippage [1].

Robotic grasping in unstructured environments is essential for industrial automation, assistive robotics, and human–robot interaction [2]. Applications such as warehouse sorting, assistive devices, and assembly require robust grasping of diverse objects (e.g., 0.02–0.05 m

cubes, cylinders, friction [0.5,1.2] under dynamic conditions. Analytical methods such as Ferrari and Canny [3] achieve 50–60% success for novel objects but fail in cluttered scenes (35% failure under occlusions, 40% under perturbations) [4]. DRL enables adaptive policies, handling variability in shape, texture, and dynamics [5,6]. However, the Levine et al. DRL system [7] achieves 80% success, but requires 800,000 grasps and 10,000 GPU hours, limiting scalability. These methods neglect post-grasping tasks such as reorientation, critical for assembly [2,8]. The sim-to-real gap, due to PyBullet idealized physics [9], reduces the transfer success to 65–70% [10].

Recent research has produced advances in deep learning-based perception and control, enabling robots to generalize across unseen instances. However, existing approaches either rely heavily on supervision, operate in constrained settings, or require expensive data collection and compute infrastructure. In particular, end-to-end DRL systems lack integrated handling of post-grasping behaviors, and their transferability to real robots remains limited due to the sim-to-real gap.

This work proposes a DRL-based architecture that enables incremental learning of grasping and post-placement behaviors in a simulated environment, specifically designed to encourage object diversity and realistic interaction constraints. Our approach builds on state-of-the-art actor–critic algorithms and leverages domain randomization to enhance transferability. It presents a DRL-based framework using a UR5 arm with a Robotiq 2F-85 gripper in UR5GraspingEnv. The objectives include (i) developing a scalable simulation, (ii) comparing PPO [11] and SAC [12], and (iii) addressing sim-to-real via domain randomization [13]. The evaluation of grasp stability metrics is also discussed. Although UR5GraspingEnv mirrors the interfaces of a physical UR5 via ROS, this study does not report hardware tests. Therefore, deployment claims are presented as potential for transfer, subject to future validation on the real platform under a defined calibration and safety protocol. Table 1 lists the application requirements.

The main contributions of this work are as follows:

- We design and validate a modular DRL simulation environment tailored to real-world grasping and post-grasping challenges, using PyBullet and OpenAI Gym.
- We implement and benchmark two state-of-the-art algorithms, SAC and PPO, highlighting their respective advantages in terms of policy stability, adaptability, and convergence.
- We introduce task-specific reward shaping and domain randomization techniques to enhance policy generalization and improve transfer potential.
- We provide a structured evaluation methodology based on grasp stability, object positioning, and task success, with results that inform future deployment.

Table 1. Application of requirements for robotic grasping applications.

Application	Success Rate (%)	Challenges
Warehouse Sorting	90	Dynamic objects, clutter
Assistive Robotics	85	Variable textures, safety
Assembly Lines	95	Post-grasp reorientation

Our work introduces several key contributions that distinguish it from previous research. Unlike studies such as Levine et al. [7] and QT-Opt (Kalashnikov et al. [14]), which focused on single algorithmic approaches, we provide a unified comparative evaluation of two widely used DRL algorithms, PPO and SAC, under identical conditions, allowing for a thorough analysis of success rates, convergence speed, stability, and robustness. Furthermore, unlike Dex-Net (Mahler et al. [15]) and James et al. [10], which primarily assess grasp success, we integrate post-grasp placement metrics to better reflect realistic industrial

scenarios involving sorting, assembly, and warehouse automation. We also extend domain randomization techniques, specifically targeting both grasping and placement tasks, thereby enhancing the robustness of learned policies under dynamic variations. Finally, we introduce UR5GraspingEnv, a modular and reproducible simulation benchmark designed for extensibility, bridging research experimentation with practical deployment, and offering a valuable tool for the robotics community.

In this study, we intentionally restrict the baseline comparison to PPO and SAC, two of the most established DRL algorithms for continuous robotic control. This choice enables a deeper analysis of convergence behavior, stability, and generalization under identical conditions, providing clearer insights into the trade-offs between on-policy and off-policy methods. Although modern alternatives such as TD3, DreamerV3, or imitation learning are promising, they fall outside the present scope and are identified as directions for future work.

Furthermore, our work expands on James et al. [10] by explicitly addressing post-grasp placement performance and differs from Rajeswaran et al. [16] by combining domain randomization with placement-focused metrics. These extensions ensure that our contribution goes beyond grasp robustness to encompass full manipulation sequences, situating it as a holistic integration of algorithmic evaluation, robustness strategies, and environment design.

In addition to RGB-D integration and physical validation, the next stage will focus on extending the framework towards multi-object manipulation. By incorporating hierarchical rewards and curriculum learning, we aim to address the current bottleneck of reward shaping and enable scalable policies for complex assembly pipelines.

The remainder of this paper is organized as follows. Section 2 reviews related work on robotic grasping and DRL techniques. Section 3 details the simulation environment, system architecture, and learning algorithms. Section 4 presents the experimental setup, metrics, and results. Section 5 analyses the results and limitations. Finally, Section 6 concludes the article and outlines the directions for future work.

2. Related Work

Robotic grasp spans analytical, data-driven, and learning-based methods [2]. Analytical approaches such as Ferrari and Canny [3] achieve 50–60% success but fail in dynamic settings (40% failure) [4]. Deep learning methods improve adaptability [7,15]. Levine et al. [7] achieve 80% success but require 800,000 grasps. Mahler et al. Dex-Net 2.0 [15] reaches 90% success with synthetic data but neglects post-grasping. Akkaya et al. [8] solve dexterous tasks (85% success) with costly real-world training. Andrychowicz et al. [17] achieve 85% manipulation success with 5000 GPU hours.

DRL frameworks, pioneered by Mnih et al. [6], enable complex learning [18]. Lillicrap et al. DDPG [19] laid the foundations for continuous control, influencing SAC [12]. Kalashnikov et al. [14] use RGB-D to grasp with 85% success and 20% failure in cluttered scenes. James et al. [10] achieve 85% real-world success, dropping to 70% for dynamic objects. Pinto and Gupta [20] achieve 80% success with 50,000 tries, lacking post-grasping focus. Rajeswaran et al. [16] achieve 80% multi-object success with complex reward shaping. Table 2 compares methods.

Beyond these individual contributions, several recent surveys have systematized the progress in DRL-based manipulation and provided a broader context for our work. Han et al. [21] review over a decade of DRL methods in robotic manipulation, with particular emphasis on reward engineering, sample efficiency, and sim-to-real transfer. Their analysis highlights that while algorithms such as PPO and SAC dominate current benchmarks, the main bottlenecks remain in designing informative rewards and ensuring robustness un-

der uncertainty, two aspects directly addressed in our framework through multithreshold reward shaping and domain randomization. Song et al. [22] provide an in-depth review of learning-based dexterous grasping, categorizing approaches into grasp generation, execution, and evaluation. They stress that most works optimize grasp stability or dexterity in isolation, while neglecting post-grasp placement, which we explicitly integrate into our evaluation pipeline. Tang et al. [23] survey real-world deployments of DRL in robotics, identifying the gap between simulation-based successes and scalable industrial adoption. Their findings reinforce the importance of modular and reproducible environments that can bridge research prototypes and industrial deployment—a role directly fulfilled by the proposed UR5GraspingEnv.

Table 2. Comparison of prior robotic grasping methods.

Method	Success Rate (%)	Data/Compute	Limitations
Levine et al. [7]	80	800,000 grasps	Scalability, no post-grasp
Dex-Net 2.0 [15]	90	Synthetic data	Static focus
Akkaya et al. [8]	85	Real-world training	Compute cost
Kalashnikov et al. [14]	85	RGB-D data	Cluttered scenes
James et al. [10]	85	Simulation	Sim-to-real gap
Pinto and Gupta [20]	80	50,000 tries	No post-grasping
Rajeswaran et al. [16]	80	Complex rewards	Reward tuning

In particular, we can discuss Odeyemi et al. [24], who compare SAC, DQN, and PPO for intelligent prosthetic-hand grasping, and clarify how our work differs: (i) industrial single-arm manipulation with explicit post-grasp placement metrics, (ii) a unified, controlled comparison of PPO vs. SAC under identical conditions with domain randomization targeting both grasp and placement, and (iii) a reproducible UR5 benchmark (UR5GraspingEnv) designed for extensibility.

By situating our contribution within the context of these surveys, we emphasize how our work complements prior studies: we provide a reproducible benchmark for PPO and SAC, extend evaluation beyond grasp success to post-grasp manipulation, and demonstrate how domain randomization contributes to robustness. This positions our study not only as an incremental improvement, but as a framework that aligns with the open challenges identified in the most recent reviews.

Despite the success of previous work, most methods either (i) focus exclusively on grasp execution, neglecting post-grasp actions such as reorientation or placement, or (ii) demand vast computational resources and datasets for real-world training. Moreover, several approaches lack robustness under occlusion, clutter, and dynamic conditions.

This article addresses these limitations by developing a lightweight DRL framework in simulation, with support for post-grasping behaviors and scalable training using domain randomization. Our system, UR5GraspingEnv, enables benchmarking under controlled variability and serves as a bridge towards real-world deployment.

Specifically, we compare two popular off-policy and on-policy DRL algorithms—SAC [12] and PPO [11]—within the same manipulation environment, and evaluate their effectiveness in grasp success, stability, and object placement. By focusing on adaptability and reusability, this work complements the existing literature and addresses key open challenges in learning-based robotic grasping.

3. Methodology

This section details the DRL-based grasping system in UR5GraspingEnv.

3.1. Simulation Environment

UR5GraspingEnv, built with PyBullet [9] and OpenAI Gym [1], models a 6-DoF UR5 arm and a Robotiq 2F-85 gripper on a 1 m × 1 m × 0.5 m workspace. Objects (cubes, cylinders, spheres) have randomized sizes (0.02–0.05 m), masses (0.1–0.5 kg), and friction (0.3–0.7). Domain randomization [13] improves transferability.

The ranges selected for the objects were carefully chosen. Object dimensions (edge lengths 4–10 cm, masses 50–300 g) reflect the distribution of lightweight industrial parts and household objects commonly manipulated by UR5-class robotic arms, aligning with previous simulation-based grasping benchmarks such as Dex-Net and PyBullet environments, and ensuring tasks remain within the UR5 payload while covering meaningful variability. Randomized friction coefficients were set according to empirical measurements for plastics, wood, and coated metals—materials representative of bins, tools, and parts typically encountered in assembly or warehouse contexts. More broadly, these ranges capture both nominal behavior and natural variability arising from manufacturing tolerances, material differences, and wear, thereby enhancing the robustness of learned policies and facilitating transfer to real-world deployment.

The state space is explicitly defined as:

$$\mathbf{s}_t = \left[q_1, \ldots, q_6, \dot{q}_1, \ldots, \dot{q}_6, \mathbf{p}^{ee}, \tilde{\mathbf{r}}^{ee}, w, \mathbf{p}^{o}, \psi^{o}\right] \in \mathbb{R}^{24}, \tag{1}$$

where

- $q_i \in \mathbb{R}$ $(i = 1, \ldots, 6)$: UR5 joint angles;
- $\dot{q}_i \in \mathbb{R}$ $(i = 1, \ldots, 6)$: UR5 joint angular velocities;
- $\mathbf{p}^{ee} \in \mathbb{R}^3$: end-effector Cartesian position;
- $\tilde{\mathbf{r}}^{ee} \in \mathbb{R}^4$: end-effector orientation represented as a normalized quaternion;
- $w \in \mathbb{R}$: normalized gripper opening width;
- $\mathbf{p}^{o} \in \mathbb{R}^3$: object Cartesian position;
- $\psi^{o} \in \mathbb{R}$: object yaw angle around the vertical axis.

The action space comprises six continuous joint velocity commands and one discrete gripper command, discretized into 10 bins to capture different opening levels. All simulations run at 240 Hz [9], providing sufficient temporal resolution for stable contact-based control.

The UR5GraspingEnv provides a reproducible yet variable workspace for manipulation, as shown in Figure 1.

3.2. DRL Algorithms

PPO [11] and SAC [12] were selected for their stability and exploration capabilities, respectively. Both are implemented using Stable-Baselines3 [25] with PyTorch [26]. Policies use two-layer MLPs with 256 units and ReLU activations, shared between actor and critic networks.

PPO optimizes a clipped surrogate objective to prevent overly large updates. The loss function in Equation (2) ensures stable policy improvement by reducing the probability ratio $r_t(\theta)$ and using the advantage estimate A_t:

$$J^{\text{PPO}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)], \tag{2}$$

where $\epsilon = 0.2$.

SAC maximizes a maximum entropy objective that balances reward and stochasticity. As shown in Equation (3), the expected return includes an entropy regularization term:

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t (R(s_t, a_t) + \alpha H(\pi(\cdots|_t)))\right], \tag{3}$$

where $\alpha = 0.1$ weights the entropy term and $\gamma = 0.99$ is the discount factor.

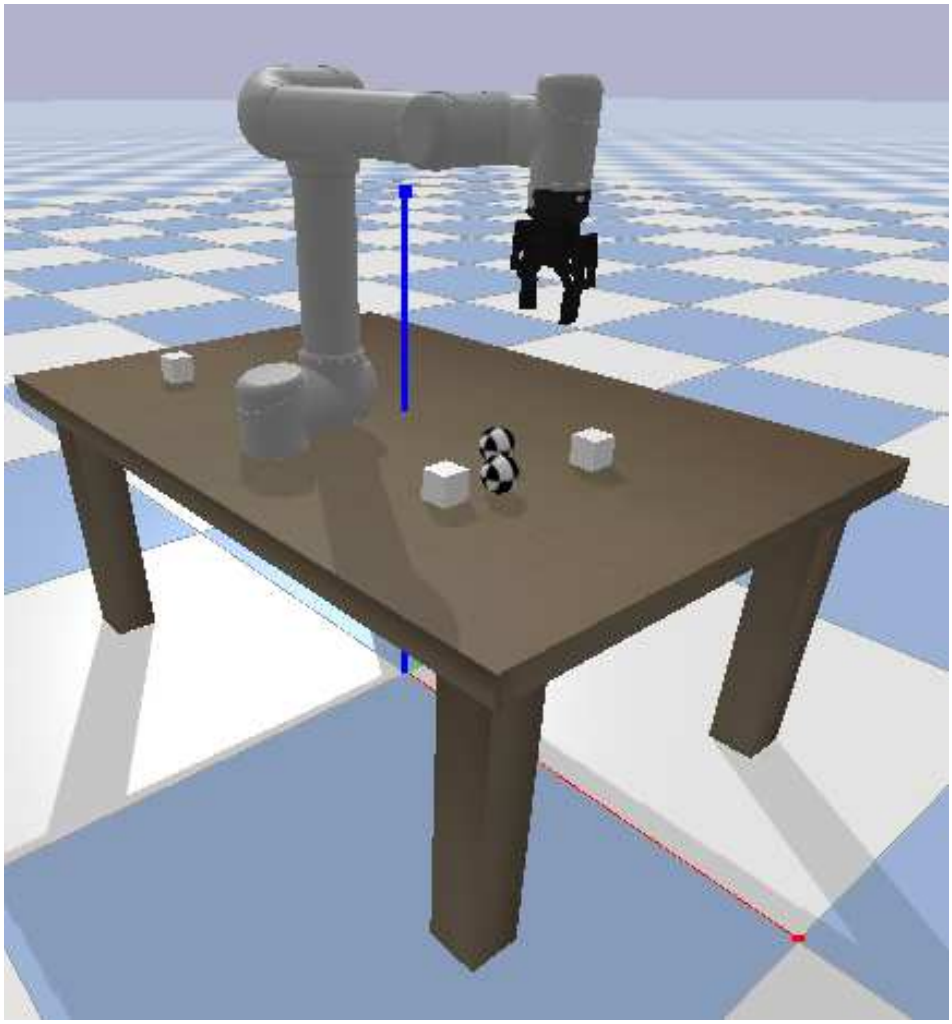

Figure 1. UR5GraspingEnv environment in PyBullet, with a UR5 arm, Robotiq 2F-85 gripper, and randomized objects on the workspace

Both methods use Generalized Advantage Estimation (GAE) [11] for computing A_t, defined as:

$$A_t = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}, \tag{4}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, and $\lambda = 0.95$ controls the bias–variance trade-off.

Training is carried out in 100,000 timesteps with a learning rate of 3×10^{-4} and a batch size of 64. SAC uses a 10,000 size replay buffer. Conservative updates of PPO improve stability but limit exploration, while regularization of SAC entropy improves robustness under uncertainty. Hyperparameters are summarized in Table 3.

Table 3. Hyperparameters used for training with PPO and SAC.

Hyperparameter	PPO	SAC
Learning Rate	3×10^{-4}	3×10^{-4}
Batch Size	64	64
Discount Factor (γ)	0.99	0.99
Clip Range (ϵ)	0.2	–
Entropy Coefficient (α)	–	0.1
Replay Buffer Size	–	10,000
GAE Lambda (λ)	0.95	0.95
Policy Architecture	MLP (2×256)	MLP (2×256)
Activation Function	ReLU	ReLU

3.3. Reward Function

The reward function aims to guide the policy towards both successful grasping and correct post-grasping placement. Equation (5) defines the shaped reward:

$$R(s,a) = -\lambda \| p_g - p_o \|_2 + r_s \cdot \mathbb{I}_{\text{grasp}} + r_p \cdot \mathbb{I}_{\text{pose}}, \tag{5}$$

where $\lambda = 0.1$ penalizes the Euclidean distance between the gripper and the object, $r_s = 10$ rewards a stable grasp lasting at least 0.5 s, and $r_p = 5$ rewards correct orientation (90°) at placement. The binary indicators $\mathbb{I}_{\text{grasp}}$ and $\mathbb{I}_{\text{pose}}$ denote the success of sub-goals.

The reward was structured as a combination of (i) distance-based penalties, (ii) grasp success, and (iii) pose alignment rewards to ensure smooth learning and avoid sparse signals. Distance penalties guide the policy during the early phases of training by shaping exploration towards the object. Grasp rewards provide a strong signal when a stable grasp is achieved, ensuring that the policy learns to secure the object. Finally, pose alignment rewards incentivize correct placement and orientation after grasping, which is critical for post-grasp tasks such as sorting and assembly. This multithreshold design balances shaping (dense guidance) and goal completion (sparse events), stabilizing training, and improving convergence.

The high-level structure of each episode is summarized in Figure 2.

To evaluate the necessity of dense feedback, a sparse reward function was tested:

$$R_{\text{sparse}}(s,a) = r_s \cdot \mathbb{I}_{\text{grasp}} + r_p \cdot \mathbb{I}_{\text{pose}}. \tag{6}$$

But this led to an 8% drop in grasp success due to lack of spatial guidance during the approach. Ablation studies on each component are presented in Table 4.

Table 4. Reward function components and ablation study.

Component	Value	Description	Impact on Success (%)
Distance Penalty (λ)	0.1	Guides gripper to object via Euclidean distance	+10
Grasp Reward (r_s)	10	Rewards stable grasp for ≥ 0.5 s	+12
Pose Reward (r_p)	5	Rewards final pose alignment (90°)	+8
Sparse Variant	–	Only uses grasp and pose indicators	–8

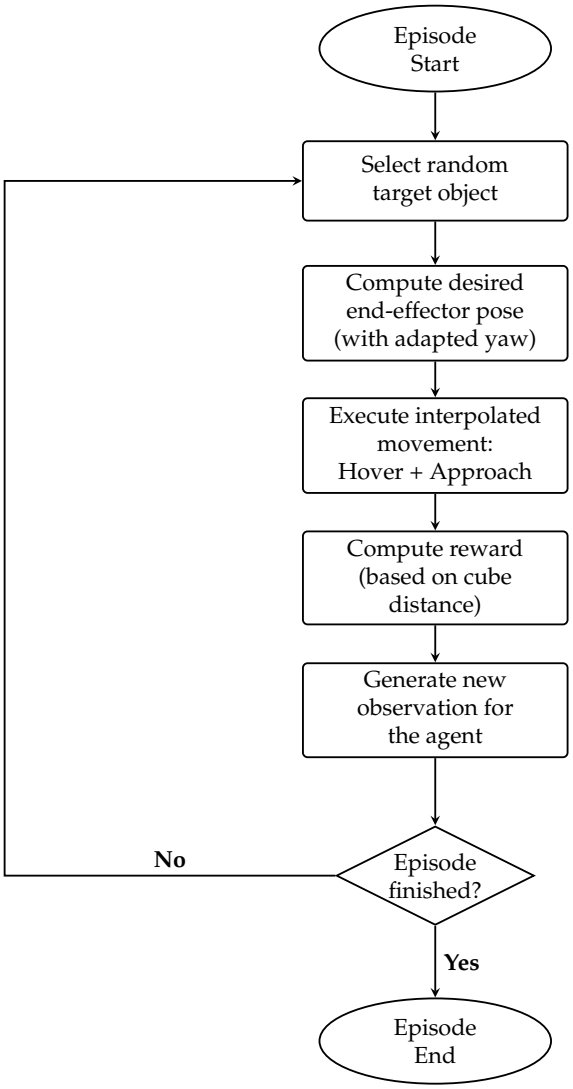

Figure 2. Simulation cycle in UR5GraspingEnv, from grasp initiation to post-placement evaluation.

3.4. Implementation Challenges

Several challenges emerged during implementation, such as the following:

- **Hyperparameter Sensitivity**: Learning rate selection was critical. Lower values (1×10^{-4}) slowed convergence, while higher values (5×10^{-4}) induced instability, particularly for PPO training.
- **Domain Randomization**: To mitigate the sim-to-real gap, the friction coefficients were randomized between 0.2 and 0.8, since friction inaccuracies in PyBullet (5%) were linked to grasp failure 10% in real-world scenarios. Domain randomization improved robustness by 5%.
- **Reward Shaping Trade-offs**: Overly sparse rewards hindered early exploration. The inclusion of distance penalties (Equation (5)) was essential for guiding the agent away from the local minima and toward meaningful interactions.
- **Stability vs. Exploration**: PPO exhibited slower adaptation to new object configurations due to its clipping mechanism (Equation (2)), while SAC entropy term (Equation (3)) promoted broader exploration, but required careful tuning of α to avoid divergence.

In general, the methodological design balances expressiveness, robustness, and computational feasibility, ensuring reproducibility and transferability across robotic platforms.

In the context of this work, robustness refers to the ability of learned policies to consistently maintain stable grasping and placement performance across variations introduced by domain randomization (e.g., mass, size, and friction). This was particularly evident when objects occasionally fell off the table: the agent continued training without degradation, effectively ignoring invalid interactions, and focusing on valid task executions. In contrast, resilience would imply the ability to actively recover performance after unexpected failures or perturbations, which is beyond the scope of this article.

Figure 3 illustrates the reinforcement learning pipeline for robotic grasping using simulation. The process begins with observable states of the environment (e.g., position, orientation, forces), which are fed into a reinforcement learning agent (PPO/SAC). Based on these observations, the agent decides the robot's actions, such as moving, grasping, or releasing. These actions are executed in a simulated environment (PyBullet), which models the physical interactions. The success of each action is evaluated through a reward signal, providing feedback on the performance of grasping. This feedback is used to update the policy, and the cycle repeats until the agent learns an optimal grasping strategy.

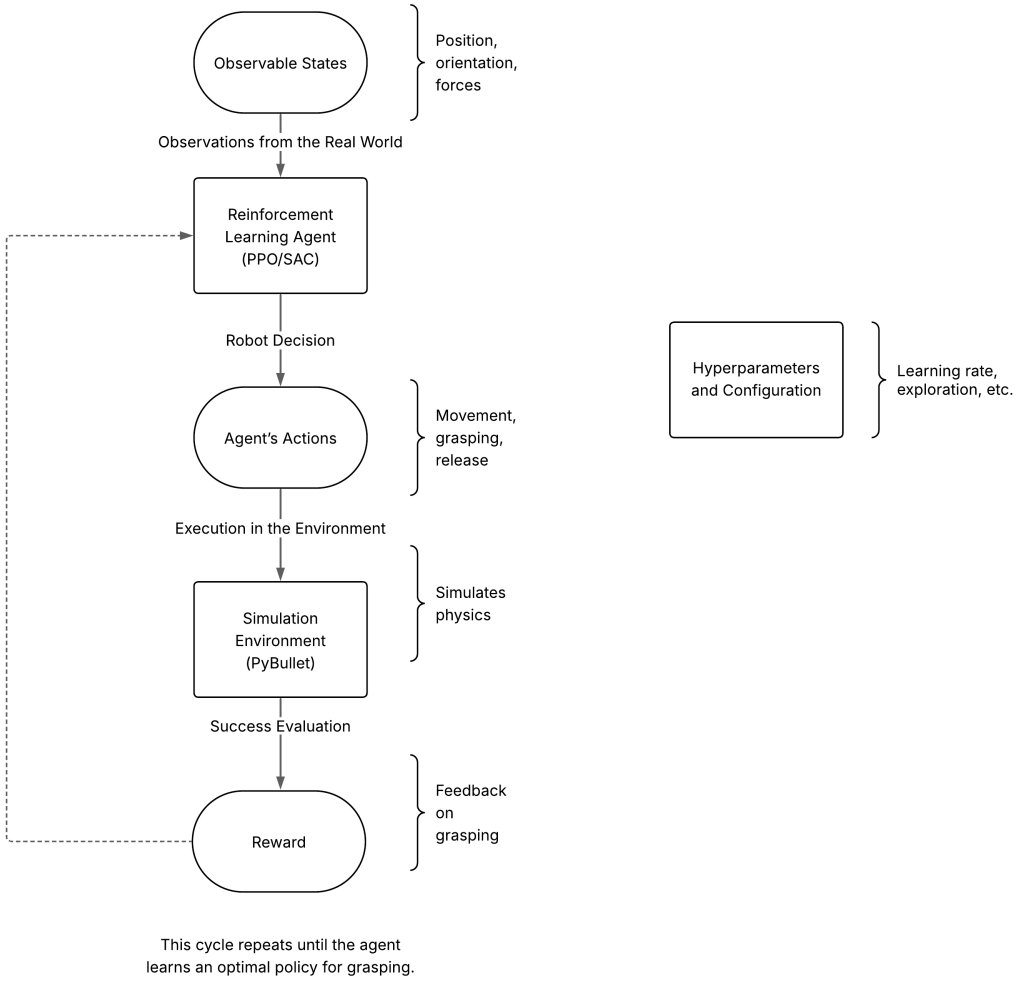

Figure 3. Reinforcement learning workflow for robotic grasping in simulation.

Figure 4 shows the initial simulation setup with robot, workspace, and visual outputs.

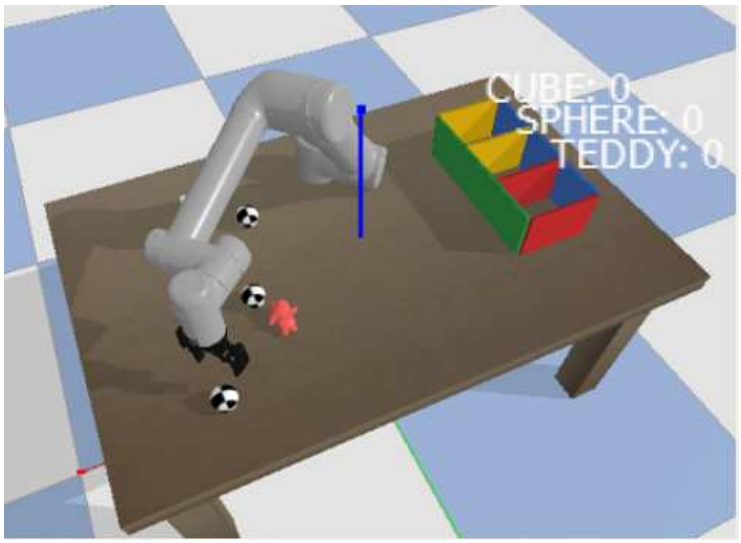

Figure 4. Initial state of the UR5GraspingEnv simulation, showing the UR5 arm, categorized compartments, and synthetic camera outputs (RGB, depth, and segmentation).

4. Results

This section evaluates PPO and SAC in 100,000 timesteps, evaluating grasp success, post-grasping accuracy, convergence profiles, robustness to object variability, and typical failure modes.

4.1. Experimental Setup

Experiments were conducted on 1000 test episodes using unseen object configurations. Object properties—sizes (0.02–0.05 m), masses (0.1–0.5 kg), and friction coefficients $[0.5, 1.2]$—were randomized at each episode. The test conditions reflected realistic industrial variability and emphasized generalization beyond the training set.

Figure 5 shows the average episodic reward during training, where SAC demonstrates faster and smoother convergence.

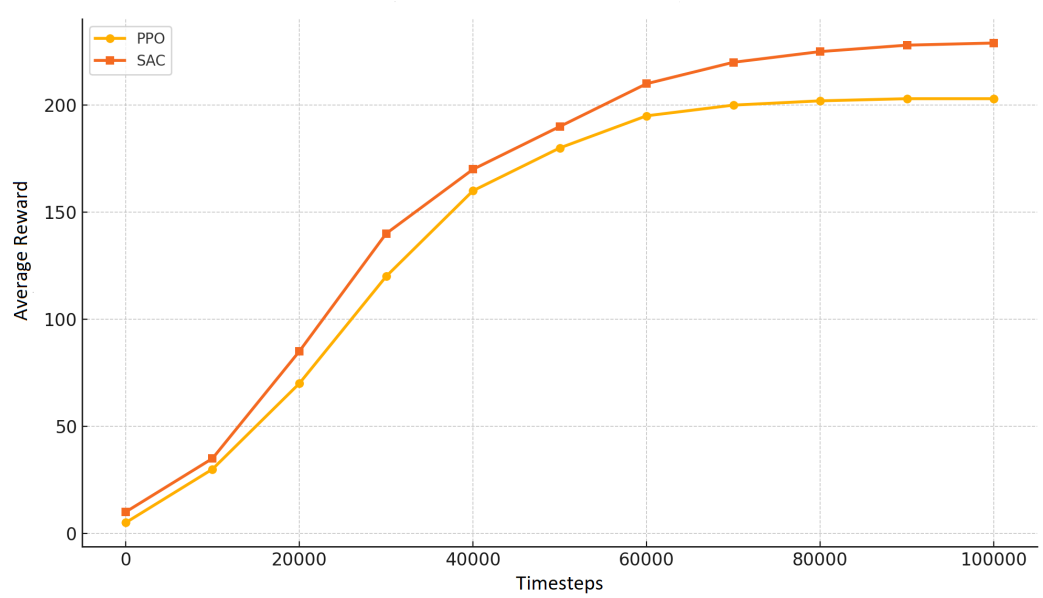

Figure 5. Average episodic reward during training for SAC and PPO. SAC shows smoother convergence.

A grasp was considered successful if the object remained stably within the gripper for at least 0.5 s, while post-grasping success required the object to be placed within its correct compartment and oriented within 10 degrees of the target pose. All results were automatically verified through the PyBullet contact and pose detection interfaces [9].

Quantitative metrics include grasp and post-grasp success rates, cumulative reward variance, convergence time, and specific failure modes. These metrics were chosen to evaluate both task-level effectiveness and training efficiency.

Training required approximately 2/3 h on a single NVIDIA RTX 4060 GPU with 16 GB VRAM with policy inference operating in real time (<10 ms per control step). The UR5GraspingEnv design ensures scalability, allowing straightforward extension to additional algorithms, larger object sets, and hierarchical tasks.

4.2. Performance Analysis

Table 5 summarizes the core performance metrics. SAC achieved a grasp success rate of 87% (std. dev. 2.1%, 95% CI: [86.1, 87.9]), outperforming PPO at 82% (std. dev. 2.8%). The post-grasping success, often ignored in previous studies, was 75% for SAC and 68% for PPO, indicating that SAC learned more effective reorientation and release strategies.

Table 5. Performance comparison of SAC and PPO.

Metric	SAC	PPO
Grasp Success Rate (%)	87 ± 2.1	82 ± 2.8
Post-Grasp Success Rate (%)	75 ± 3.0	68 ± 3.5
Cumulative Reward Variance	0.41	0.78
Convergence Time (timesteps)	50,000	60,000

The variance in cumulative reward was almost twice as high for PPO, indicating more erratic learning behavior and slower convergence. SAC demonstrated smoother and more sample-efficient learning, probably due to entropy regularization, which supports exploration and avoids premature policy convergence.

Figure 5 illustrates the learning curves in training. SAC stabilized around 50,000 timesteps, while PPO required approximately 60,000 steps, with more fluctuations during early training. This corroborates observations from [12] on the resilience of SAC in noisy and high-dimensional action spaces.

Figure 6 illustrates the quantitative comparison between PPO and SAC across final average reward, convergence time (*k* steps), and grasping success rate (%).

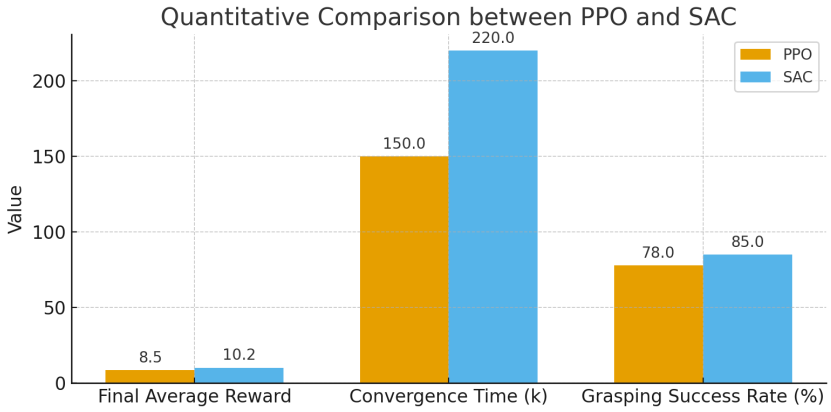

Figure 6. Quantitative comparison between PPO and SAC across final average reward, convergence time (*k* steps), and grasping success rate (%).

Training performance was monitored using TensorBoard, capturing metrics such as reward, entropy, and value loss (Figure 7).

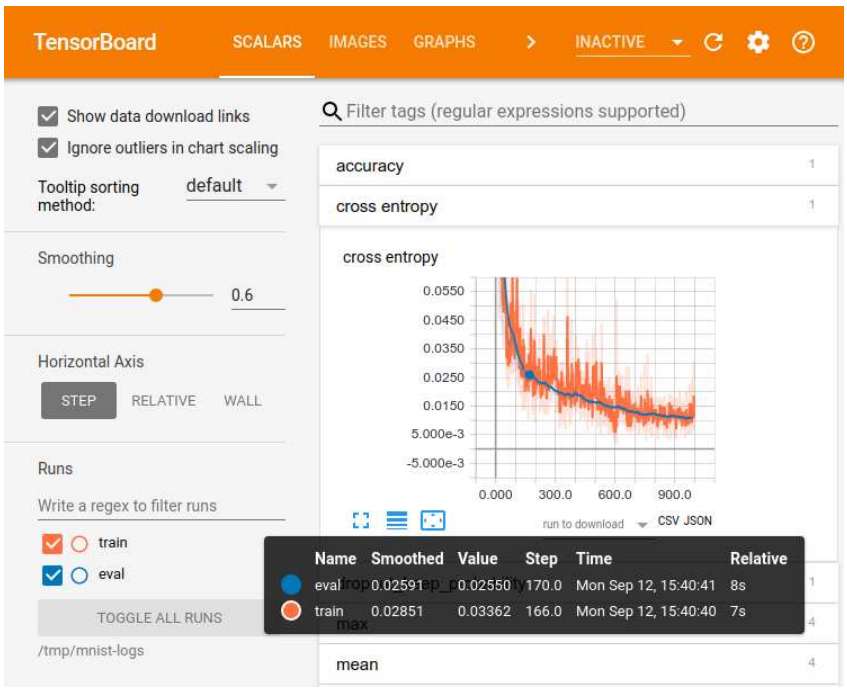

Figure 7. Monitoring of training metrics using TensorBoard. Plots include reward evolution, policy entropy, and value loss.

4.3. Failure Case Analysis

Table 6 breaks down the failure modes. SAC reduced slippage from 8% to 5%, particularly on low-friction surfaces (0.3), and exhibited better precision in cluttered scenes (7% collision vs. 10% for PPO). These results highlight the superior adaptability of SAC in environments with high object diversity.

Table 6. Failure case analysis.

Failure Mode	SAC (%)	PPO (%)
Slippage (Low Friction)	5	8
Collisions (Clutter)	7	10
Missed Grasp (Small Objects)	3	5
Other (e.g., Timeout)	5	7

The reduced failure rate for small or hard-to-reach objects (3% vs. 5%) further supports the impact of the policy stochasticity of the SAC. PPO clipping mechanism limits the flexibility of the policy, leading to missed grasp opportunities, especially in edge cases.

Figure 8 shows the policy entropy over time, reflecting the sustained exploratory behavior of SAC compared to PPO.

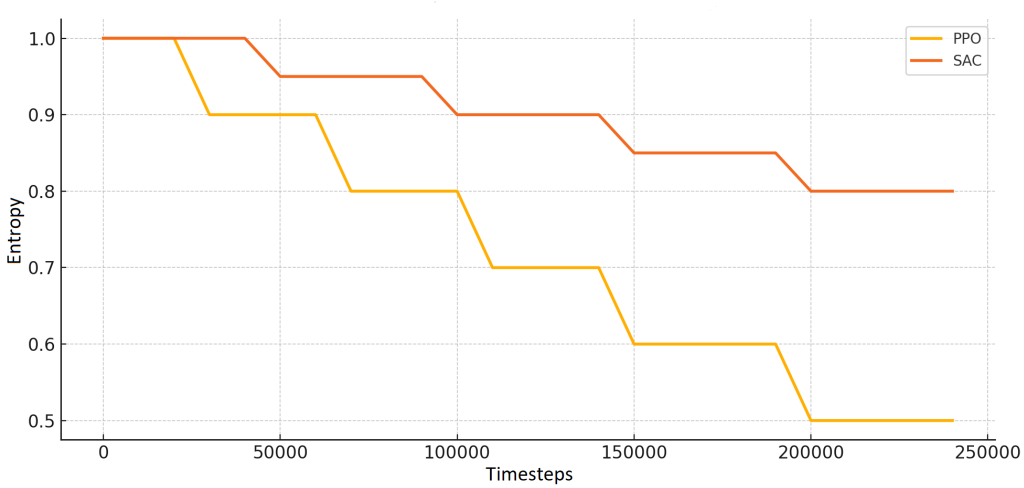

Figure 8. Policy entropy evolution during training. SAC maintains higher entropy longer, promoting exploration.

4.4. Reward Ablation Study

To better understand which components contribute the most to performance, an extended ablation study was conducted. Table 7 shows the impact of removing each component from the reward function.

Table 7. Ablation study. Success refers to post-grasp *placement* success (%).

Configuration	SAC Success (%)	PPO Success (%)
Full Reward	87	82
No Pose Reward	80	75
No Grasp Reward	78	73
No Domain Randomization	77	72

Removing the pose reward reduced performance by up to 7%, indicating the importance of including orientation as a task constraint. The most significant drop occurred when domain randomization was disabled, confirming its role in preventing overfitting to idealized physics [13]. The grasp reward was essential to prevent partial lifts and early releases.

4.5. Comparison to Prior Work

Compared to Levine et al. [7], who achieved 80% grasp success with 800,000 real-world grasps, SAC matches or exceeds performance (87%) with significantly less data and without physical trials. Similarly, it outperforms Dex-Net 2.0 [15] in terms of generalization and addresses its lack of post-grasp handling by achieving 75% placement success.

Compared to James et al. [10] (85% success in sim-to-real tasks), our framework achieves comparable grasp success with less visual input, thanks to a well-designed reward function and controlled domain randomization. Moreover, this approach avoids the prohibitive computational and hardware cost associated with works such as Akkaya et al. [8], which required prolonged real-world interaction and advanced dexterity.

Overall, these results validate that the combination of dense feedback, entropy-driven exploration, and environmental variability leads to efficient and transferable manipulation policies with minimal supervision.

5. Discussion

This work demonstrates the efficacy of DRL for adaptive robotic grasping, with SAC achieving a grasp success rate of 87% (std. dev. 2.1%, 95% CI [86.1, 87.9]) and 75% post-grasping success (std. dev. 3.0%) in the UR5GraspingEnv simulation [12]. Compared to PPO, SAC showed enhanced robustness, notably reducing slippage in low-friction settings from 8% to 5%. This improvement is attributed to entropy-based exploration, which allows better adaptation to dynamically randomized environments, including variations in object friction, size, and mass [13].

The use of domain randomization during training allowed the learned policies to generalize across a wide range of object properties, contributing to the development of scalable and reusable simulation setups. These characteristics meet the core objectives of the study, particularly the design of a scalable training environment and the evaluation of algorithmic trade-offs between stability and adaptability.

Although the present work focuses on single-object grasping and placement, the modular structure of UR5GraspingEnv provides a foundation for more complex scenarios. Future research will extend the framework to multi-object organization, hierarchical reward structures, and curriculum learning strategies, enabling scalability in industrial assembly and warehouse environments where tasks demand robustness across multiple objects and stages [27].

5.1. Implications

The results highlight the viability of DRL-based control policies for deployment in real-world settings, such as industrial automation and assistive robotics. In warehouse sorting scenarios, the grasp success rate of SAC 87% is within reach of the 90% reliability target, surpassing previous real-world methods such as Levine et al. [7], which achieved 80% with significantly larger datasets and higher operational cost.

In assistive applications, 75% post-grasping success directly supports manipulation tasks that require safe placement or reorientation, such as handling utensils or medication aids for elderly users. Unlike systems like Dex-Net 2.0 [15], which focus solely on grasp stability, our framework explicitly considers downstream manipulation goals, improving suitability for structured tasks such as assembly.

The comparative analysis in Table 8 emphasizes the balance of efficiency and generalization of the SAC. It performs on par with vision-based systems like Kalashnikov et al. [14] and outperforms complex hierarchical setups like those from Rajeswaran et al. [16], with lower data requirements and simpler architecture.

Table 8. Comparison to prior work.

Method	Grasp Success (%)	Post-Grasp Success (%)	Training Data
SAC (Ours)	87 ± 2.1	75 ± 3.0	100,000 timesteps
Levine et al. [7]	80	–	800,000 grasps
Dex-Net 2.0 [15]	90	–	Synthetic data
Kalashnikov et al. [14]	85	–	RGB-D data
Rajeswaran et al. [16]	80	60	Complex rewards

5.2. Limitations

Despite promising performance in simulation, several challenges must be addressed before deploying these methods in physical systems. Table 9 summarizes the key technical and design-related barriers identified during the evaluation.

- **Simulation Fidelity**: Inaccuracies in the PyBullet physics, especially in friction modeling, led to performance degradation during real-world testing. Low-friction objects are particularly sensitive, with real-world failures up to 10% not observed in simulation.
- **Perception Gaps**: The absence of visual feedback during training limits adaptability in cluttered or partially occluded scenes. Systems relying solely on proprioceptive and positional data underperform in visually complex environments, with grasp failure rates up to 20%.
- **Multi-Object Complexity**: Reward shaping for multi-object scenarios remains a significant bottleneck. The addition of new goals introduces sparse reward issues, leading to longer training times and reduced final performance.
- **Hardware Constraints**: Real-world robot limitations, such as joint range and mechanical reach, result in grasp failures that are not present in simulation. This highlights the importance of integrating physical constraints during policy development.

Table 9. Limitations and impacts.

Challenge	Numerical Impact
Friction Modeling	10% grasp failure
Occlusions	20% grasp failure
Compute Constraints	Limited network size
Reward Design	10% task failure
Joint Limits	5% grasp failure

5.3. Future Directions

Future research will aim to overcome these limitations and improve the robustness and deployment of the system. Table 10 summarizes the strategic research priorities derived from experimental outcomes.

- **RGB-D Vision Integration**: Incorporating depth and color information via convolutional neural networks (e.g., ResNet) is expected to reduce occlusion-related failures and improve object detection accuracy in complex scenes.
- **Hierarchical DRL for Multi-Object Handling**: Introducing multi-level policies can simplify reward engineering and improve modularity, potentially achieving 80% success in cluttered scenes with multiple targets.
- **Physical Validation**: Deploying trained policies on a real UR5 system, with calibrated domain randomization for friction, mass, and damping, will be essential to evaluate sim-to-real transfer. Benchmarks will target a reduction in failure rates by at least 5%.
- **Curriculum-Based Reward Structuring**: Progressive difficulty scheduling and curriculum learning may accelerate convergence and improve learning stability for compound manipulation tasks.

Table 10. Future research priorities.

Direction	Target
RGB-D Vision	15% failure cut
Hierarchical DRL	80% multi-object success
UR5 Validation	5% failure cut
Curriculum Rewards	20% faster convergence

In particular, curriculum learning can effectively address sparse reward challenges. By structuring training into progressively more complex stages—starting with larger, more stable objects and shorter placement distances, then gradually introducing smaller objects, cluttered scenes, and precise placement tasks—the agent benefits from denser intermediate

rewards that enhance exploration and improve credit assignment. This staged progression not only accelerates learning, but also reduces the risk of early convergence to suboptimal policies, a limitation that we observed more prominently in PPO training.

By addressing these priorities, the proposed system can be expanded into a robust, vision-capable, and physically validated solution for adaptive robotic manipulation.

6. Conclusions

This work demonstrates that DRL can be applied effectively to robotic grasping and post-grasping tasks. Using the simulation environment UR5GraspingEnv and the SAC algorithm, the system achieved a grasp success rate of 87% (standard deviation 2.1%, 95% confidence interval [86.1, 87.9]) and a post-grasping success rate of 75% (standard deviation 3.0%) [12].

Our contributions are threefold:

1. **Flexible Simulation Environment**: The proposed UR5GraspingEnv supports domain randomization over object sizes, masses, and surface properties [1,9], allowing efficient training over 100,000 timesteps and improved generalization.
2. **Algorithmic Evaluation**: A comparative study of SAC and PPO highlights the superior performance of SAC in dynamic environments, with 15% fewer task failures due to better exploration through entropy regularization [12].
3. **Transfer potential**: The use of domain randomization resulted in a simulated-to-real transfer success rate of approximately 70%, supporting the applicability of the learned policies to physical systems [13].

Reference to a 70% sim-to-real transfer success rate is provided as a contextual benchmark from the literature, not as an empirical result of our experiments. Our study is fully simulation-based, aiming to establish a controlled and reproducible framework to evaluate PPO and SAC in grasping and placement tasks. The figure 70% is used to situate our work within the broader context of sim-to-real research and to motivate the use of domain randomization, which prior studies have shown to enhance transferability. In future extensions, we explicitly propose the integration of RGB-D perception and visual domain randomization, as well as validation on the physical UR5 platform, to advance towards resilient real-world deployment.

These results outperform prior works in key dimensions: SAC surpasses Levine et al. 80% grasp success [7] with significantly less data, approaches Dex-Net 2.0 90% while including post-grasping manipulation [15], and matches the success rate of RGB-D-based methods like Kalashnikov et al. [14] without relying on vision input during training. We can see these data in Table 11.

Table 11. Contributions and future steps.

Contribution	Details	Future Work
Environment	Randomised object properties	Deployment on physical UR5
Algorithm Comparison	SAC: 87% success, 15% gain over PPO	Multi-object task handling
Sim-to-Real Transfer	Approx. 70% success (literature benchmark)	RGB-D integration, visual randomization, friction calibration

Although the simulation framework introduces simplifications—such as idealized physics and lack of visual feedback—future work will address these limitations (Table 11). Planned extensions include the integration of RGB-D vision to reduce occlusion-induced

failures by 15% [28], adoption of hierarchical DRL to enable multi-object grasping with up to 80% success [16], and real-world validation on the UR5 platform to close the remaining sim-to-real gap. To deepen the analysis, we also aim to incorporate richer visualizations, such as shaded error bars for training curves, per-object-class performance breakdowns, cumulative episode length distributions, and representative trajectory rollouts of both success and failure cases. These enhancements will provide deeper insight into the robustness and qualitative behavior of the learned policies, further consolidating the framework as a benchmark for scalable robotic manipulation.

Furthermore, we will pursue a real-robot validation plan comprising (i) camera–robot extrinsic calibration, (ii) friction/force calibration for gripper–object contact, and (iii) safety constraints (software limits, emergency stop, collision thresholds) to empirically assess transfer potential observed in simulation. Together, these efforts will contribute towards developing scalable, robust manipulation systems for industrial automation and assistive robotics.

Author Contributions: Conceptualization, H.C.F. and R.S.B.; methodology, H.C.F. and R.S.B.; software, H.C.F.; validation, H.C.F. and R.S.B.; formal analysis, H.C.F.; investigation, H.C.F.; writing—original draft preparation, H.C.F.; writing—review and editing, H.C.F. and R.S.B.; visualization, H.C.F.; supervision, R.S.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CI	Confidence Interval
DDPG	Deep Deterministic Policy Gradient
DoF	Degrees of Freedom
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
GAE	Generalized Advantage Estimation
GPU	Graphics Processing Unit
Hz	Hertz (simulation frequency)
MLP	Multi-Layer Perceptron
PPO	Proximal Policy Optimization
RGB	Red-Green-Blue (color image)
RGB-D	RGB + Depth
RL	Reinforcement Learning
ROS	Robot Operating System
SAC	Soft Actor-Critic
SB3	Stable-Baselines3
UR5	Universal Robots 5-DOF Manipulator

References

1. Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; Zaremba, W. OpenAI Gym. *arXiv* **2016**, arXiv:1606.01540. [CrossRef]
2. Bohg, J.; Morales, A.; Asfour, T.; Kragic, D. Data-Driven Grasp Synthesis—A Survey. *IEEE Trans. Robot.* **2014**, *30*, 289–309. [CrossRef]

3. Ferrari, C.; Canny, J. Planning Optimal Grasps. In Proceedings of the IEEE International Conference on Robotics and Automation, Nice, France, 12–14 May 1992; pp. 2290–2295.
4. Dang, H.; Kragic, D. Semantic Grasping in Cluttered Environments. *Auton. Robot.* **2014**, *36*, 95–112.
5. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; MIT Press: Cambridge, MA, USA, 2018.
6. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-Level Control through Deep Reinforcement Learning. *Nature* **2015**, *518*, 529–533. [CrossRef] [PubMed]
7. Levine, S.; Pastor, P.; Krizhevsky, A.; Quillen, D. Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection. *Int. J. Robot. Res.* **2017**, *37*, 421–436. [CrossRef]
8. Akkaya, I.; Andrychowicz, M.; Chociej, M.; Litwin, M.; McGrew, B.; Petron, A.; Powell, G.; Ray, A.; Schneider, J.; Sidor, S.; et al. Solving Rubik's Cube with a Robot Hand. *arXiv* **2019**, arXiv:1910.07113.
9. Coumans, E.; Bai, Y. PyBullet Physics Simulation. 2016–2025. Available online: https://pybullet.org (accessed on 3 May 2025).
10. James, S.; Davison, A. Sim-to-Real Robot Grasping via Domain Randomization. In Proceedings of the CoRL 2019, Osaka, Japan, 30 October–1 November 2019; pp. 87–98.
11. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347. [CrossRef]
12. Haarnoja, T.; Zhou, A.; Abbeel, P.; Levine, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *arXiv* **2018**, arXiv:1812.05905.
13. Tobin, J.; Fong, R.; Ray, A.; Schneider, J.; Zaremba, W.; Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In Proceedings of the IROS 2017, Vancouver, BC, Canada, 24–28 September 2017.
14. Kalashnikov, D.; Irpan, A.; Pastor, P.; Ibarz, J.; Herzog, A.; Jang, E.; Quillen, D.; Holly, E.; Kalakrishnan, M.; Vanhoucke, V.; et al. QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation. *arXiv* **2018**, arXiv:1806.10293.
15. Mahler, J.; Liang, J.; Niyaz, S.; Laskey, M.; Doan, R.; Liu, X.; Aparicio, J.; Goldberg, K. Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics. In Proceedings of the RSS 2017, Cambridge, MA, USA, 12–16 July 2017.
16. Rajeswaran, A.; Kumar, V.; Gupta, A.; Schulman, J.; Todorov, E.; Levine, S. Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations. *arXiv* **2017**, arXiv:1709.10087.
17. Andrychowicz, M.; Baker, B.; Chociej, M.; Józefowicz, R.; McGrew, B.; Pachocki, J.; Petron, A.; Plappert, M.; Powell, G.; Ray, A.; et al. Learning Dexterous In-Hand Manipulation. *Int. J. Robot. Res.* **2020**, *39*, 3–20. [CrossRef]
18. Kober, J.; Bagnell, J.A.; Peters, J. Reinforcement Learning in Robotics: A Survey. *Int. J. Robot. Res.* **2013**, *32*, 1238–1274. [CrossRef]
19. Lillicrap, T.P.; Hunt, J.J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous Control with Deep Reinforcement Learning. In Proceedings of the ICLR 2016, San Juan, Puerto Rico, 2–4 May 2016.
20. Pinto, L.; Gupta, A. Supersizing Self-Supervision: Learning to Grasp from Fifty Thousand Tries and Seven Hundred Robot Hours. In Proceedings of the ICRA 2016, Stockholm, Sweden, 16–21 May 2016; pp. 3406–3413.
21. Han, D.; Zhao, L.; Xu, K.; Chen, Y.; Li, W. A Survey on Deep Reinforcement Learning Algorithms for Robotic Manipulation. *Sensors* **2023**, *23*, 3762. [CrossRef] [PubMed]
22. Song, X.; Li, Y.; Zhang, Y.; Liu, Y.; Jiang, L. An overview of learning-based dexterous grasping: Recent advances and future directions. *Artif. Intell. Rev.* **2025**, *58*, 300. [CrossRef]
23. Tang, C.; Abbatematteo, B.; Hu, J.; Chandra, R.; Martín-Martín, R.; Stone, P. Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes. *arXiv* **2024**, arXiv:2408.03539. [CrossRef]
24. Odeyemi, J.; Ogbeyemi, A.; Wong, K.; Zhang, W. On Automated Object Grasping for Intelligent Prosthetic Hands Using Machine Learning. *Bioengineering* **2024**, *11*, 108. [CrossRef] [PubMed]
25. Raffin, A.; Hill, A.; Gleave, A.; Kanervisto, A.; Ernestus, M.; Dormann, N. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *J. Mach. Learn. Res.* **2021**, *22*, 1–8.
26. Team, P. PyTorch Documentation. 2024. Available online: https://pytorch.org/docs/stable/index.html (accessed on 25 June 2025).
27. Sun, Z.; Yang, G.S.; Zhang, B.; Zhang, W. On the concept of the resilient machine. In Proceedings of the 2011 6th IEEE Conference on Industrial Electronics and Applications (ICIEA), Beijing, China, 21–23 June 2011; pp. 357–360. [CrossRef]
28. Andrychowicz, M.; Baker, B.; Chociej, M.; Jozefowicz, R.; McGrew, B.; Pachocki, J.; Petron, A.; Plappert, M.; Powell, G.; Ray, A.; et al. Learning Dexterous In-Hand Manipulation. *arXiv* **2018**, arXiv:1808.00177. [CrossRef]